As 3 Grandes Categorias de Desafios Técnicos(e entrevistas técnicas)

A maior parte das vezes vão te perguntar o O QUÊ É, então basta decorar o que são os conceitos abaixo.

A exceção é o Live Coding, onde querem que mostre como implementar eles Ao Vivo(Com a tela compartilhada), é questão de memorização também, dessa vez, digitando o código que memorizou.

Podem te fazer perguntas extras para preencher o tempo da reunião, novamente, memorização prévia.

1. Senso Comum para DEVs

Padrões de Projeto(Design Patterns)

- O que é uma classe singleton? Por que não deveria utilizar mais?

- Singleton

- Factory Method

- Abstract Factory- Observer

- Strategy

- Builder

- Decorator

- Adapter

Arquiteturas

- Qual a diferença entre Arquitetura de software e design patters?

- Clean Architecture

- Hexagonal(Ports & Adapters)

- Package-by-feature

- MVC

- SAGA

- MVC

- MVVM

- CQRS(Command and Query Responsibility Segregation)

- EDA(Event Driven)

- Message Broker

Princípios de Desenvolvimento de Software

- SOLID

- DRY (Don't Repeat Yourself)

- KISS (Keep It Simple, Stupid)

- YAGNI (You A't Gonna Need It)

- Programação Defensiva

Testes de Software

- Testes unitários

- Testes de integração

- Testes de sistema

- Testes de aceitação

- TDD (Test-Driven Development) Nunca fale mal disso. Veja um desafio de live coding que exige TDD

Passo a Passo Completo com o Sandro Mancuso, autor do livro The Software Craftsmanship

- Mocking e Stubbing o louco tô falando de você agora

Controle de Versão

- Conceitos básicos de Git

- Fluxo de trabalho com Git (branching, merging, rebasing)

- GitFlow

- Trunk Based

- Estratégias de commit e pull request

Live Coding

Palíndromo, é uma palavra ou frase que se lê da mesma forma de trás para frente, tem algumas variações que provavelmente vai encontrar no Live Coding.

Se concentre primeiro em memorizar o código para esse desafio, pois **ele tem uma frequência mais alta que os demais.**

Se encontrar durante o Teste em uma plataforma(HackerRank, etc.), basta tirar print e pedir pro Gepeto resolver, não perca tempo com isso, pois NUNCA vai precisar resolver um desses no dia a dia do trabalho.

1. Palíndromos com Pontuação e Espaços Ignorados

Esta é a variação mais comum. A frase é considerada um palíndromo se as letras formam uma sequência espelhada, ignorando espaços, pontuação e diferenças entre maiúsculas e minúsculas.

Exemplo: "Anotaram a data da maratona."

2. Palíndromos Numéricos

Números também podem ser palíndromos.

Exemplo: 12321

3. Palíndromos de Caracteres

Em vez de palavras completas, a sequência de caracteres individuais pode formar um palíndromo.

Exemplo: "racecar"

4. Palíndromos Fraseológicos

Frases inteiras podem ser palíndromos, embora sejam mais raros.

Exemplo: "Socorram-me, subi no ônibus em Marrocos."

5. Palíndromos com Restrições

Pode haver desafios que exigem palíndromos com certas restrições, como usar apenas um conjunto específico de letras ou ter um comprimento mínimo.

Podem pedir para detectar um palíndromo no meio de uma frases, e pedir pra exibir o maior Palíndromo ou o menor Palíndromo.

6. Palíndromos Bidimensionais

Em contextos mais avançados, você pode encontrar palíndromos em duas dimensões, como em quadrados de letras onde as linhas e colunas são palíndromos.

Dicas para Implementar a Verificação de Palíndromos em Código:

Limpeza do Input: Remova espaços, pontuação e converta tudo para minúsculas (ou maiúsculas) para simplificar a comparação.

Comparação direta: Compare a primeira metade da string com a segunda metade invertida.

Uso de estruturas de dados: Pilhas ou filas podem ser úteis para inverter a string e facilitar a comparação.

Recursão: Para desafios mais complexos, querem que você use a recursão, novamente basta decorar previamente como fazer isso.

FizzBuzz

Esse é fácil de decorar, consiste em usar um condicional pra verificar se um número de uma lista é impar ou par.

PigLatin

Esse é mais chato de resolver, consiste em manipulação de string de acordo com uma regra que te passarem. Decore previamente para não passar vergonha.

Group By com Streams

```
public static List<String> findTopNFrequentWords(List<String>
wordsToBeChecked, int topMost) {
   final var lowerCaseWords =
wordsToBeChecked.stream().map(String::toLowerCase).toList();


lowerCaseWords.stream().sorted().distinct().limit(topMost).toList();

   return lowerCaseWords.stream()
```

```
            .filter(Objects::nonNull)
            .map(String::toLowerCase)
            .collect(Collectors.groupingBy(w -> w,
Collectors.counting()))
            .entrySet()
            .stream()
            .sorted(Comparator
                    .comparing(Map.Entry<String, Long>::getValue,
Comparator.reverseOrder())
                    .thenComparing(Map.Entry::getKey))
            .limit(topMost)
            .map(Map.Entry::getKey)
            .collect(Collectors.toList());
}
```

👉 Desafios Técnicos e Live Codings

Princípios de Banco de Dados

- Normalização e desnormalização

- Chaves primárias e estrangeiras

- **ACID(Atomicidade, Consistência, Isolamento, Durabilidade)**

- Índices e otimização de consultas

- Views e stored procedures

Redes e Protocolos

- Modelo OSI e TCP/IP

- HTTP/HTTPS

- WebSockets

- RESTful APIs(Verbos HTTP GET POST PUT DELETE PATCH HEAD), quais são idempotentes

- SOAP

- GraphQL

- Segurança em redes (SSL/TLS, VPNs, firewalls)

- Rate Limit(**Token Bucket,** Leak Bucket)

DevOps e CI/CD

- Integração contínua (CI)

- Entrega contínua (CD)

- Docker e containers

- Kubernetes

- Monitoramento e logging

Algoritmos e Estruturas de Dados

- Algoritmos de ordenação (quick sort, merge sort, etc.)

- Algoritmos de busca (binary search, breadth-first search, depth-first search)

- Listas, Pilhas, Filas

- Big O

- Árvores (árvores binárias, AVL, B-trees)

- Grafos (representação, travessia, algoritmos de caminho mais curto)

Manipulação de Strings

Inverter uma string:

- **Problema**: Dada uma string, retorne a versão invertida dela.
- **Exemplo**: Entrada: "hello", Saída: "olleh".
- **Abordagem**: Pode ser resolvido utilizando o método reversed em várias linguagens, ou transformando a string em uma lista, invertendo e unindo os elementos.

Contar vogais e consoantes:

- **Problema**: Dada uma string, conte quantas vogais e consoantes ela possui.
- **Exemplo**: Entrada: "hello", Saída: vogais: 2, consoantes: 3.
- **Abordagem**: Verifique cada caractere e classifique-o como vogal ou consoante.

Remover caracteres duplicados:

- **Problema**: Dada uma string, remova caracteres duplicados.
- **Exemplo**: Entrada: "programming", Saída: "progamin".
- **Abordagem**: Use uma estrutura de dados, como um conjunto, para armazenar apenas caracteres únicos.

Encontrar o primeiro caractere não repetido:

- **Problema**: Encontre o primeiro caractere que não se repete em uma string.
- **Exemplo**: Entrada: "swiss", Saída: "w".
- **Abordagem**: Armazene a contagem de cada caractere, depois percorra a string novamente para identificar o primeiro que aparece uma única vez.

Verificar anagramas:

- **Problema**: Verifique se duas strings são anagramas (contêm as mesmas letras na mesma quantidade).
- **Exemplo**: Entradas: "listen" e "silent", Saída: true.
- **Abordagem**: Ordene ambas as strings e compare-as, ou conte a frequência dos caracteres.

Compactar uma string (compressão RLE):

- **Problema**: Comprima uma string, contabilizando caracteres repetidos consecutivos.
- **Exemplo**: Entrada: "aabccccccaaa", Saída: "a2b1c5a3".
- **Abordagem**: Percorra a string contando repetições e monte a string comprimida.

Substituir caracteres:

- **Problema**: Substitua todas as ocorrências de um caractere específico em uma string.
- **Exemplo**: Entrada: "banana", substitua "a" por "o", Saída: "bonono".
- **Abordagem**: Pode ser feito com métodos de substituição da linguagem, como replace em Python e Kotlin.

Permutação de String: Dada uma string, gere todas as permutações possíveis de seus caracteres.

Exemplo: Entrada: "abc", Saída: ["abc", "acb", "bac", "bca", "cab", "cba"].

Abordagem: A geração de permutações geralmente requer uma abordagem recursiva. Em linguagens como Python e Kotlin, bibliotecas padrão já oferecem métodos de permutação (por exemplo, itertools.permutations em Python).

Em Kotlin, pode-se escrever uma função recursiva para permutar os caracteres ou usar a função kotlin.collections.permutations() disponível para coleções.

Programação Funcional

- Funções de alta ordem

- Imutabilidade(em quê ela favorece cenários multithread)

- Lazy evaluation

- Map, reduce, filter

Segurança em Desenvolvimento

- OWASP Top Ten

- XSS (Cross-Site Scripting)

- CSRF (Cross-Site Request Forgery)

- SQL Injection

- Práticas de autenticação e autorização (OAuth, JWT, etc.)

Integração de Sistemas

- Mensageria (RabbitMQ, Kafka)

- Serviços de integração (ESB, API Gateway)

- Webhooks

- Arquitetura orientada a eventos

Ferramentas e Tecnologias

- IDEs e editores de código (IntelliJ, VS Code)

- Ferramentas de build e automação (Maven, Gradle)

- Análise de código estático (SonarQube, ESLint)

2. Detalhes da Stack(Java)

Diferença entre StringBuffer e StringBuilder(https://g.co/gemini/share/f5537b6f95af)

- Thread-safety e sincronização(@Syncronized)

- Performance e uso apropriado

O que é String Pool

- Internação de strings em Java

- Benefícios e considerações de uso

Gerenciamento de Memória

- Heap e Stack

- Garbage Collection

- Young and Old Generation (G1GC Generacional)

- Metaspace

- Escopo de variáveis e ciclo de vida

Coleções e Estruturas de Dados

- Listas, Conjuntos, Mapas

- Estruturas de dados: Pilha, Fila, Árvore, Grafo

Concurrency e Multithreading

- Modelos de concorrência

- Problemas comuns: deadlock, livelock, starvation

- Ferramentas de sincronização

3. Systems Design

Design Uber for Us

- Arquitetura geral e componentes principais

- Serviços de localização e mapeamento

- Banco de dados e escolha de armazenamento

- Escalabilidade e balanceamento de carga

- Tolerância a falhas e alta disponibilidade

- Segurança e autenticação

Exemplo Real de System Design do PIX(https://www.youtube.com/watch?v=q6MNAxb-_RM)

P.S. É a mesma linha para entrevistas no Exterior também

Design an e-commerce

- Gerenciamento de catálogos de produtos
- Carrinho de compras e processamento de pedidos
- Integração com sistemas de pagamento
- Recomendações de produtos e personalização
- Gerenciamento de estoque e logística

Design a Social Media Platform

- Gestão de perfis de usuário
- Feed de notícias e algoritmos de recomendação
- Comunicação em tempo real (chat e mensagens)
- Gerenciamento de conteúdo e moderação
- Privacidade e controle de acesso

Padrões de Design de Sistemas Distribuídos

- Microserviços vs Monólitos

- Padrões de comunicação (REST, gRPC, mensagens)

- Tolerância a falhas e recuperação(Resiliência)

- Monitoramento e logging

- Circuit Breaker(Disjuntor)